



Network Anomaly Detection with Kitsune Framework

Mini Project Report

Deep Learning Course Project

Submitted To:

Faculty, Department of AI & Data Engineering

Kodem Jyothesh Kumar Dora

2023UAI1807

Paturi Vivek Chowdary

2023UAI1814

Department of AI & Data Engineering
Malaviya National Institute of Technology, Jaipur

Abstract

This project presents a network anomaly detection system using the Kitsune Framework for identifying malicious network behavior, specifically ARP Man-in-the-Middle (MitM) attacks. The system processes raw packet capture files, extracts structured traffic features, and applies unsupervised machine learning techniques for anomaly detection.

The proposed workflow converts packet capture data into a 115-feature dataset using TShark and performs preprocessing through normalization and feature formatting. Kitsune, an ensemble-based autoencoder framework, is trained exclusively on benign traffic data and later evaluated on datasets containing both normal and malicious traffic samples.

Experimental analysis demonstrates that Kitsune can effectively distinguish anomalous network traffic from benign behavior through reconstruction error-based anomaly scoring. Threshold-based evaluation and visualization methods such as boxplots and confusion matrices further validate the effectiveness of the system for cybersecurity monitoring applications.

Keywords: Network Security, Kitsune Framework, Anomaly Detection, ARP MitM Attack, Cybersecurity, Autoencoders, Unsupervised Learning, Network Intrusion Detection, TShark, Deep Learning

1 Introduction

1.1 Background

Modern computer networks are increasingly vulnerable to cyberattacks targeting communication protocols and local network infrastructure. One common attack is the Address Resolution Protocol (ARP) Man-in-the-Middle (MitM) attack, where attackers intercept communication between systems to monitor or manipulate sensitive information.

Traditional rule-based intrusion detection systems often fail to identify unknown or evolving attacks. Machine learning-based anomaly detection systems provide a stronger alternative by learning normal traffic behavior and detecting deviations from it.

1.2 Problem Statement

Detecting malicious traffic in real time is difficult because attack traffic may closely resemble normal communication patterns. ARP MitM attacks can remain hidden inside local networks while intercepting user traffic.

This project aims to analyze network traffic and identify anomalous behavior using the Kitsune anomaly detection framework.

1.3 Project Objectives

1. Analyze raw network traffic captured in packet format.
2. Convert packet capture data into structured datasets.

3. Extract 115 relevant network traffic features.
4. Apply the Kitsune framework for anomaly detection.
5. Detect ARP Man-in-the-Middle attacks within local networks.
6. Evaluate detection performance using multiple thresholds and metrics.
7. Demonstrate the role of machine learning in cybersecurity monitoring.

2 System Architecture

The proposed system follows a multi-stage anomaly detection pipeline.

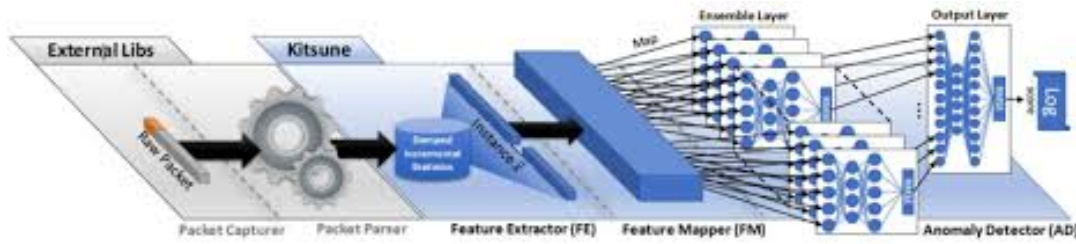
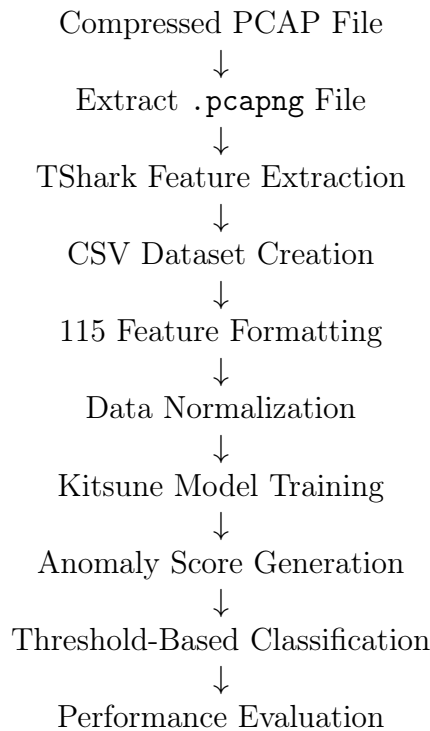


Figure 1: Kitsune Framework Architecture

2.1 Workflow Description

1. Extract compressed packet capture archives.
2. Retrieve .pcapng network traffic files.
3. Perform feature extraction using TShark.
4. Convert extracted data into CSV format.
5. Apply preprocessing and feature formatting.
6. Normalize features using MinMaxScaler.
7. Train Kitsune using benign traffic samples.
8. Evaluate the trained model on complete datasets.
9. Generate anomaly scores for network traffic.
10. Classify traffic using threshold-based detection.
11. Analyze performance using confusion matrices and visualizations.

2.2 Architecture Flow Diagram



3 Implementation

3.1 Technology Stack

- Python
- Pandas
- NumPy
- Scikit-learn
- TShark
- Kitsune Framework
- Google Colab

3.2 Core Concepts Applied

3.2.1 Autoencoder-Based Learning

Kitsune uses an ensemble of autoencoders trained to reconstruct normal traffic behavior. Malicious traffic generates higher reconstruction errors, producing larger anomaly scores.

3.2.2 Unsupervised Learning

The model is trained only on benign traffic samples. Instead of memorizing attack signatures, it learns normal traffic patterns and flags deviations as suspicious.

3.2.3 Feature Engineering

Raw packet capture traffic is transformed into structured numerical vectors containing 115 features.

3.2.4 Data Normalization

Feature values are scaled between 0 and 1 using MinMaxScaler to improve training stability and maintain balanced feature distributions.

3.2.5 Anomaly Scoring

Higher reconstruction errors indicate stronger deviations from normal traffic patterns and are treated as anomalous behavior.

3.3 Key Components Used

- **TShark:** Packet parsing and feature extraction.
- **Python Libraries:** Data preprocessing and analysis.
- **Scikit-learn:** Feature normalization.
- **Kitsune Framework:** Core anomaly detection engine.
- **Google Colab:** Cloud-based execution environment.

4 Results and Performance Analysis

4.1 Performance Overview

The trained Kitsune model was evaluated on datasets containing both benign traffic and ARP MitM attack samples. Attack traffic consistently produced higher anomaly scores than normal traffic.

4.2 Evaluation Metrics

- Accuracy
- Attack Recall
- Normal Recall
- Confusion Matrix
- Threshold-wise Performance Comparison

4.3 Threshold Analysis

Multiple thresholds including 80%, 85%, 90%, 95%, 98%, and 99% quantiles were tested.

- Lower thresholds improved attack detection sensitivity.
- Higher thresholds reduced false positive rates.
- Balanced thresholds produced stable overall performance.

4.4 Visual Analysis

A boxplot analysis of anomaly scores showed clear separation between benign and malicious traffic distributions, demonstrating the effectiveness of the Kitsune framework for anomaly detection.

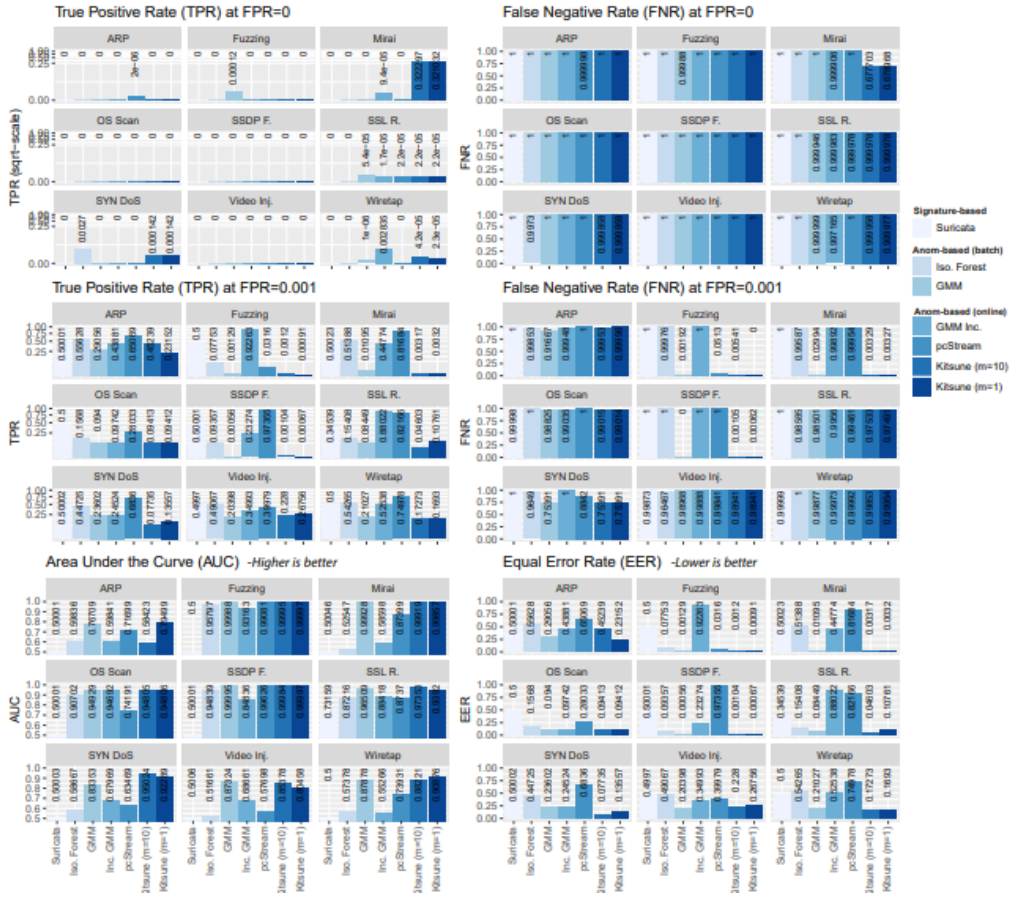


Figure 2: Experimental Results and Attack Analysis

4.5 Preliminary Conclusion

The Kitsune framework demonstrated strong capability for detecting ARP MitM attacks in real time. Proper threshold tuning improved detection accuracy while maintaining acceptable performance on normal traffic.

5 Challenges Faced and Solutions

Challenge	Solution
Handling Raw Packet Data	Used TShark for structured feature extraction from packet captures.
Maintaining Required Feature Dimensions	Applied zero-padding to ensure all records matched the 115-feature requirement.
Training Stability	Normalized all features using MinMaxScaler.
Detecting Unknown Attacks	Used unsupervised learning with Kitsune trained only on benign traffic.
Threshold Selection	Tested multiple quantile thresholds to balance recall and false positives.
Google Colab Dependency Issues	Resolved package conflicts and environment setup problems manually.

Table 1: Major challenges faced during development

6 Discussion

6.1 Advantages of Kitsune

- Effective for real-time intrusion detection.
- Detects previously unseen attacks.
- Requires only benign traffic for training.
- Lightweight compared to traditional heavy classifiers.
- Suitable for cybersecurity anomaly detection tasks.

6.2 Limitations

- Performance depends heavily on threshold tuning.
- Requires high-quality feature extraction.
- Limited evaluation on only ARP MitM attack traffic.
- Real-time deployment optimization remains future work.

7 Conclusion

This project successfully demonstrated the application of the Kitsune framework for network anomaly detection. Using unsupervised machine learning and autoencoder-based anomaly scoring, the system effectively distinguished malicious ARP MitM traffic from benign traffic.

The project highlights how machine learning techniques can strengthen cybersecurity monitoring systems by enabling real-time anomaly detection without relying on labeled attack datasets.

Future improvements may include testing on additional attack categories, real-time deployment, and integration with advanced visualization dashboards.

References

- [1] Guillem96, “kitsune-pytorch GitHub Repository,” Available at: <https://github.com/Guillem96/kitsune-pytorch>
- [2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection,” NDSS Symposium, 2018.
- [3] Pedregosa et al., “Scikit-learn: Machine Learning in Python.”
- [4] W. McKinney, “Pandas Library Documentation.”
- [5] Wireshark Foundation, “TShark Command-Line Packet Analyzer Documentation.”

A AI Tools Used

- ChatGPT
- Google Gemini

B Areas of Usage

- Coding assistance
- Debugging and dependency fixing
- Understanding Kitsune workflow
- Dataset handling
- Result interpretation
- Documentation preparation
- README generation

C Final Responsibility

Although AI tools were used during development, all final implementation decisions, testing, modifications, and verification were carried out by the project team.